



CSE461

Introduction to Robotics Lab

LAB REPORT - 2

Group : 05

Section : 06

Semester : Summer_2026

Group members :

ALENA HALDER TAMO	22301046
Tahsin Raihan Robbani	22201344
Mayen Shahriar Kabir	23101107
Mofftasim Hossain Sayem	22299109
Tahmida Tanjim Talukder	22101730

Submitted Date: 25-08-2026

Submitted to

Pratick Roy Chowdhuruy (Lecturer, BRAC University)

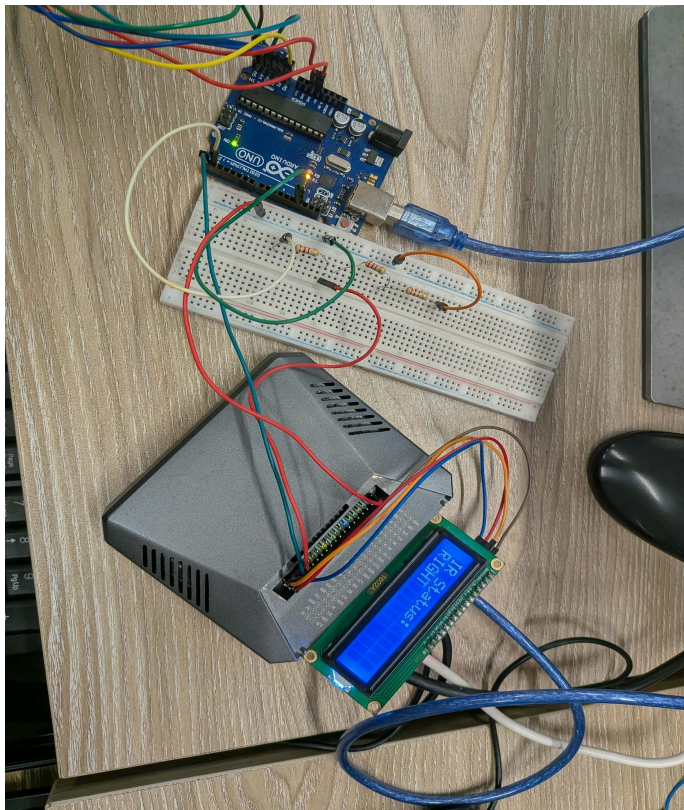
Purba Tahia Hassan (Lecturer, BRAC University)

Lab 5

1. Equipments:

- Arduino Uno R3
- Raspberry PI 4
- Analog 6-array IR sensor
- I2C LCD Display (16x2)
- Resistor (220 Ω)
- Breadboard
- Jumper wires

2. Experimental Setup:



3. Code:

```
# Enter Code Here
```

```
import serial
from gpiozero import LED
from rpi_lcd import LCD
from time import sleep

# Initialize LEDs on designated GPIO pins
led_left = LED(17)
led_center = LED(27)
led_right = LED(22)

lcd = LCD()

ser = serial.Serial('/dev/ttyS0', 9600, timeout=1)

def turn_off_all():
    led_left.off()
    led_center.off()
    led_right.off()

try:
    while True:
        if ser.in_waiting > 0:
            line = ser.readline().decode('utf-8', errors='ignore').strip()

            lcd.clear()
            lcd.text('IR Status:', 1)
            lcd.text(line, 2)

            if line == "LEFT":
                led_left.on()
                led_center.off()
                led_right.off()

            elif line == "RIGHT":
                led_left.off()
                led_center.off()
                led_right.on()

            elif line == "FORWARD":
```

```
        led_left.on()
        led_center.on()
        led_right.on()

    else:

        turn_off_all()

except KeyboardInterrupt:

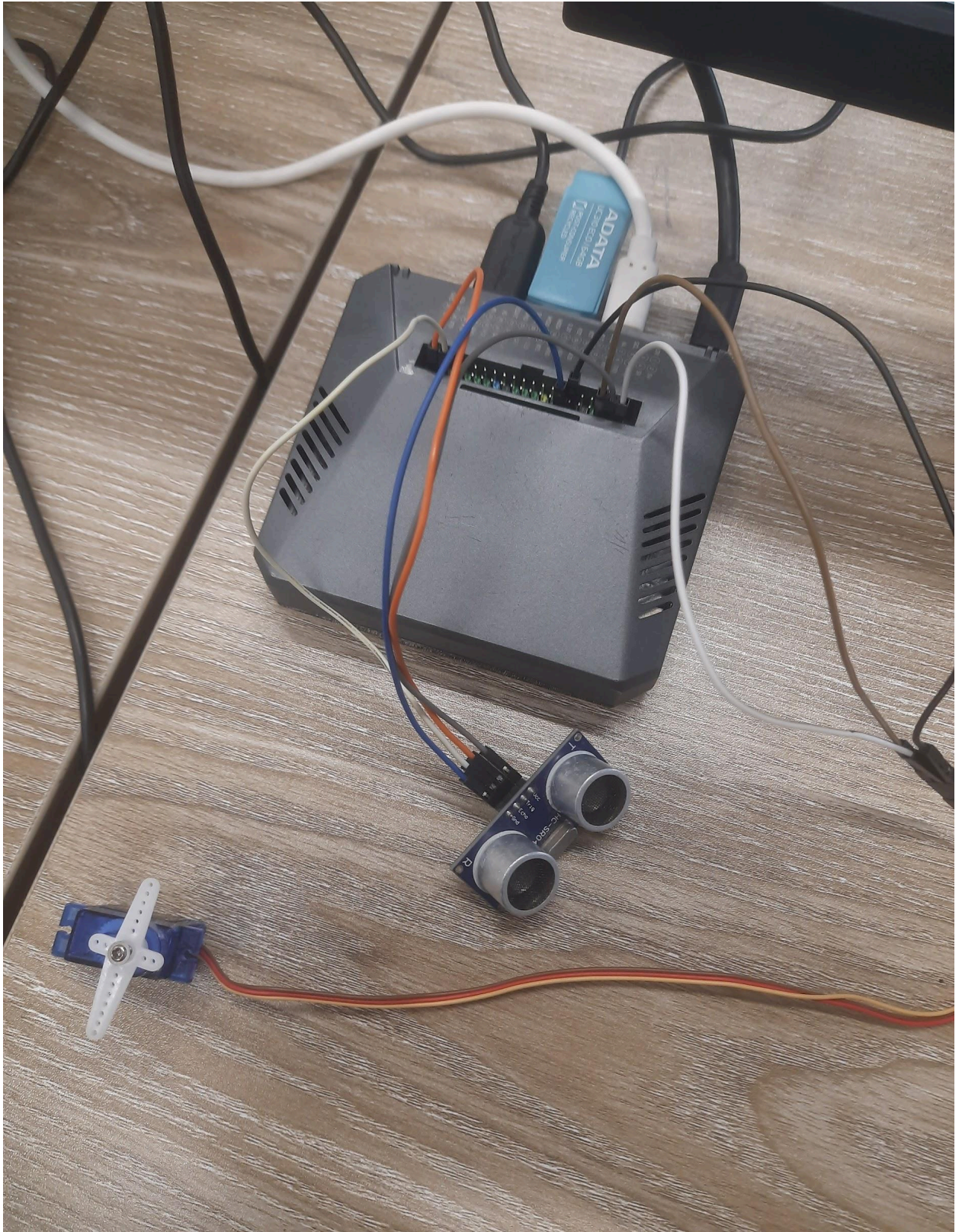
    turn_off_all()
    lcd.clear()
    ser.close()
    print("\nProgram terminated successfully.")
```

Lab 6

1. Equipments:

- Raspberry PI development board
- Servo Motor
- Ultrasonic Sensor(HC-SR04)
- Resistor (appropriate value for current limiting, typically around 220 ohms)
- Jumper wires
- Breadboard
- USB pen drive containing OS

2. Experimental Setup:



3. Code:

```
# Enter Code Here

import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)

TRIG = 21
ECHO = 20
SERVO_PIN = 17

GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.setup(SERVO_PIN, GPIO.OUT)

servo_pwm = GPIO.PWM(SERVO_PIN, 50)
servo_pwm.start(0)

def set_angle(angle):

    duty_cycle = (angle / 18.0) + 2.0
    servo_pwm.ChangeDutyCycle(duty_cycle)
    time.sleep(0.3)
    servo_pwm.ChangeDutyCycle(0)

def distance():

    GPIO.output(TRIG, False)
    time.sleep(0.2)

    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    pulse_start = time.time()
    pulse_end = time.time()

    while GPIO.input(ECHO) == 0:
        pulse_start = time.time()

    while GPIO.input(ECHO) == 1:
        pulse_end = time.time()

    pulse_duration = pulse_end - pulse_start
    dist = pulse_duration * ((343 * 100) / 2)
    return round(dist, 2)
```

```
try:
    while True:
        dist = distance()
        print(f"Measured Distance: {dist} cm")

        if dist < 5.0:
            print("Car detected (< 5cm)! Opening gate (180°)...")
            set_angle(180)
        else:
            print("Path clear (>= 5cm). Closing gate (90°)...")
            set_angle(90)

        time.sleep(0.5)

except KeyboardInterrupt:
    print("\nProgram stopped by user.")

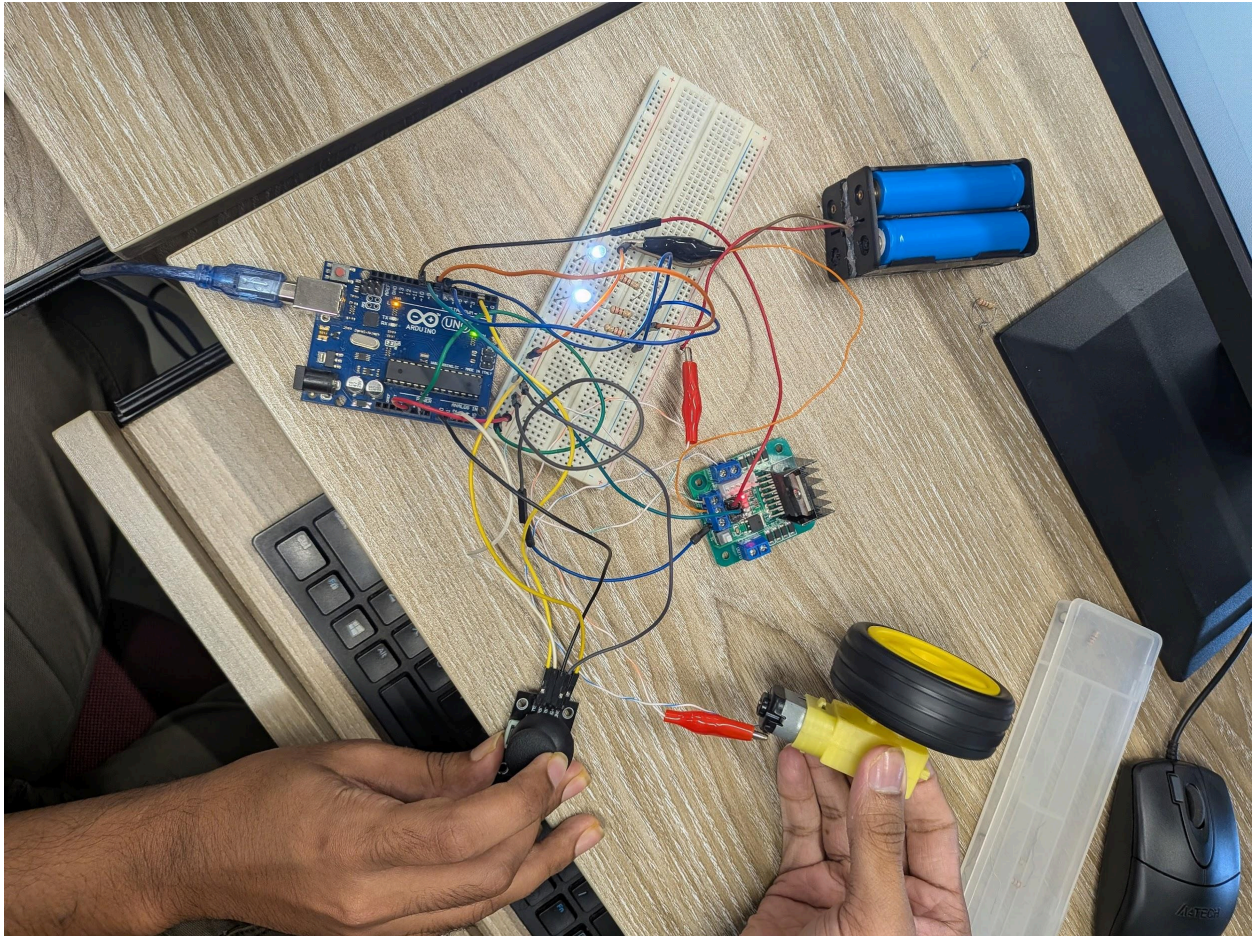
finally:
    servo_pwm.stop()
    GPIO.cleanup()
```

Lab 7

1. Equipments:

- Arduino Uno R3
- L298N Motor Driver Module
- 4-Pin Analog Joystick Module
- DC Motor (6V–9V, depending on power source)
- External Power Supply or Battery Pack (for motor)
- Breadboard
- Jumper Wires

2. Experimental Setup:



3. Code:

```
# Enter Code Here

#define VRY A1
#define threshold 200

#define IN1 10      // Motor Driver IN1
#define IN2 9       // Motor Driver IN2

#define LED_FORWARD 8 // Forward LED indicator pin
#define LED_LEFT 7    // Left LED indicator pin
#define LED_RIGHT 6   // Right LED indicator pin

void setup() {
  // Motor driver control pins set as OUTPUT
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
```

```

pinMode(LED_FORWARD, OUTPUT);
pinMode(LED_LEFT, OUTPUT);
pinMode(LED_RIGHT, OUTPUT);

Serial.begin(9600);
}

void loop() {
  int y = analogRead(VRY); // Read analog value from Y-axis

  if (y < 512 - threshold) {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);

    digitalWrite(LED_FORWARD, HIGH);
    digitalWrite(LED_LEFT, LOW);
    digitalWrite(LED_RIGHT, LOW);

    Serial.println("Forward | Forward LED: ON");
  }
  else if (y > 512 + threshold) {

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);

    digitalWrite(LED_FORWARD, LOW);
    digitalWrite(LED_LEFT, HIGH);
    digitalWrite(LED_RIGHT, HIGH);

    Serial.println("Backward | Left & Right LEDs: ON");
  }
  else {

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);

    digitalWrite(LED_FORWARD, LOW);
    digitalWrite(LED_LEFT, LOW);
    digitalWrite(LED_RIGHT, LOW);
  }
}

```

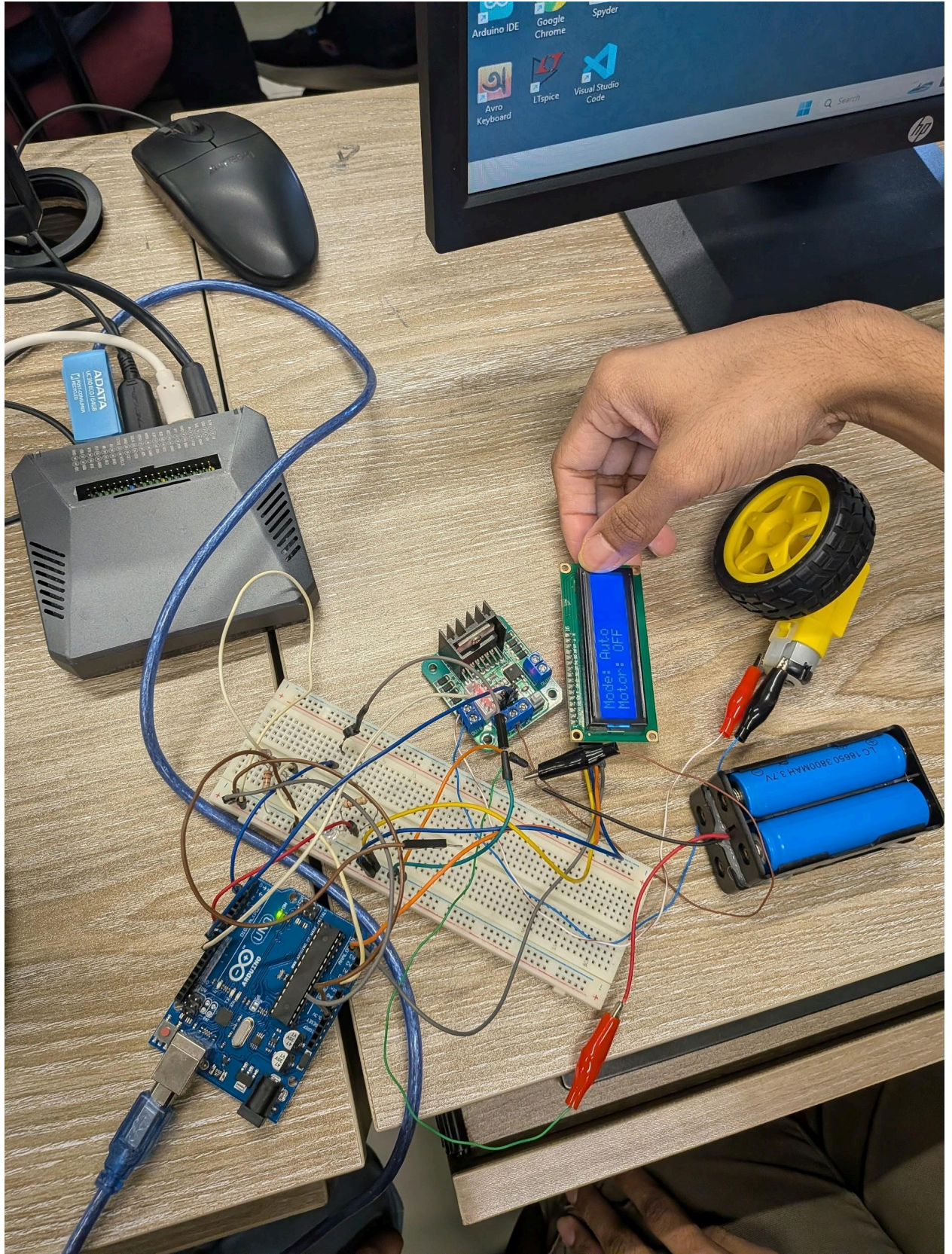
```
    Serial.println("Stop | All LEDs: OFF");  
  }  
  
  delay(300);  
}
```

Lab 8

1. Equipments:

- Arduino, Raspberry PI
- LDR
- Push Button
- DC motor
- LED
- LCD display for live status

2. Experimental Setup:



3. Code:

Enter Code Here

Arduino code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Initialize LCD at I2C address 0x27 for a 16x2 display
// (If text doesn't show, change 0x27 to 0x3F)
LiquidCrystal_I2C lcd(0x27, 16, 2);

const int LDR_PIN = A0;
const int BUTTON_PIN = 2;
const int LED_PIN = 7;
const int MOTOR_IN1 = 9;
const int MOTOR_IN2 = 8;

unsigned long lastSend = 0;

void setup() {
  Serial.begin(9600);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
  pinMode(MOTOR_IN1, OUTPUT);
  pinMode(MOTOR_IN2, OUTPUT);

  // Updated I2C LCD Initialization
  lcd.init();
  lcd.backlight();

  lcd.setCursor(0, 0);
  lcd.print("Init System...");
  delay(1000);
  lcd.clear();
}

void loop() {
  // 1. Send telemetry to Raspberry Pi every 100ms
  if (millis() - lastSend > 100) {
    int ldrVal = analogRead(LDR_PIN);
    int btnState = digitalRead(BUTTON_PIN);

    Serial.print(ldrVal);
    Serial.print(",");
    Serial.println(btnState);
    lastSend = millis();
  }

  // 2. Receive commands from Raspberry Pi
  if (Serial.available() > 0) {
```

```

String data = Serial.readStringUntil('\n');
data.trim();

int commaIndex = data.indexOf(',');
if (commaIndex > 0) {
    String modeStr = data.substring(0, commaIndex);
    int motorCmd = data.substring(commaIndex + 1).toInt();

    bool motorOn = (motorCmd == 1);
    digitalWrite(LED_PIN, motorOn ? HIGH : LOW);

    if (motorOn) {
        digitalWrite(MOTOR_IN1, HIGH);
        digitalWrite(MOTOR_IN2, LOW);
    } else {
        digitalWrite(MOTOR_IN1, LOW);
        digitalWrite(MOTOR_IN2, LOW);
    }

    // Update Display
    lcd.setCursor(0, 0);
    lcd.print("Mode: ");
    lcd.print(modeStr);
    lcd.print("    ");

    lcd.setCursor(0, 1);
    lcd.print("Motor: ");
    lcd.print(motorOn ? "ON " : "OFF");
    lcd.print("    ");
}
}
}

```

Raspberry Pi code:

```

import serial
import time

SERIAL_PORT = '/dev/ttyACM0' # Use '/dev/ttyUSB0' if using a clone Arduino
BAUD_RATE = 9600
LIGHT_THRESHOLD = 100        # Adjust (0-1023) based on ambient room light

def main():
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        time.sleep(2) # Wait for Arduino auto-reset on serial connection
        print("Connected to Arduino successfully.")
    except Exception as e:
        print(f"Error opening serial port: {e}")
        return

    manual_mode = False
    last_btn_state = 1 # INPUT_PULLUP defaults to 1 (HIGH) when unpressed

```

```

while True:
    try:
        if ser.in_waiting > 0:
            line = ser.readline().decode('utf-8', errors='ignore').strip()
            parts = line.split(',')

            if len(parts) == 2:
                ldr_val = int(parts[0])
                btn_reading = int(parts[1])

                # Falling-edge detection (triggers once per button press:
                1 -> 0)

                if btn_reading == 0 and last_btn_state == 1:
                    manual_mode = not manual_mode
                    print(f"Mode switched to: {'Manual' if manual_mode
else 'Auto'}")

                # Update state tracking AFTER checking transition
                last_btn_state = btn_reading

                # System Control Logic
                if manual_mode:
                    motor_on = 1 # Forced ON in Manual Mode
                    mode_str = "Manual"
                else:
                    motor_on = 1 if ldr_val > LIGHT_THRESHOLD else 0
                    mode_str = "Auto"

                # Send command back to Arduino
                command = f"{mode_str},{motor_on}\n"
                ser.write(command.encode('utf-8'))

    except KeyboardInterrupt:
        print("\nShutting down controller...")
        ser.write(b"Auto,0\n") # Safely turn off motor on exit
        ser.close()
        break
    except Exception as e:
        print(f"Execution Error: {e}")

if __name__ == "__main__":
    main()

```

Questions

1. In Experiment 5, The code sets `binary[i] = (sensorValues[i] > threshold) ? 0 : 1;`, and the sensor gives a high voltage on white surfaces and a low voltage on black. Figure out which binary value (0 or 1) a sensor produces when it is sitting over the black line, and use that to explain why "all six sensors = 1" is treated as STOP. [Lab 5]

Answer:

The IR sensor generates a voltage (high) on a white surface and a voltage (low) on a black surface. The code sets,

`binary[i] = (sensorValues[i] > threshold) ? 0 : 1;`

On the given arduino code the threshold is set at 500. Therefore,

If `sensorValues[i] > 500`, the `binary[i] = 0`, the surface will be white.

If `sensorValues[i] ≤ 500`, then `binary[i] = 1`, which means the surface is black.

Therefore, if a sensor is above the black line the binary output will be 1.

If all six sensors are 1 then the code will print "STOP". Since 1 represents black, all six sensors being 1 means that all six sensors are detecting the black area at the same time. This means that the robot is not just following a thin line, but at a full black, or a stopping area, and the program sees this as STOP.

2. The joystick rests around 512, and the code uses a threshold of 200. Work out the range of `analogRead` values that the code still treats as "Center" (no movement). Then explain in one sentence why having this neutral band is useful rather than reacting to every tiny change. [Lab 7]

Answer:

The code is only triggered when the reading is outside of the
lower range `= [512-200=312 ]` and
upper range `[512+200=712]`

and therefore values within this range (inclusive) are categorized as 'Center.

This neutral band is important because potentiometers on a joystick aren't actually 512, there's a little noise and a small amount of inaccuracy in the electrical readings, so moving the stick to the "neutral" position, the motor shouldn't jitter, or else it could trigger false Forward/Backward/Left/Right commands.

3. A servo expects a PWM signal at 50 Hz. Show that 50 Hz corresponds to a 20 ms period. Then explain which property of the PWM signal actually determines the angle the servo turns to and use that to explain how two PWM signals at different frequencies can still move the servo to the same angle [Lab 6]

Answer:

We know,

$$T=1/f$$

$$=1/50 \text{ Hz}$$

$$=0.02 \text{ s}$$

$$=20 \text{ ms}$$

So, 50 Hz corresponds to a 20 ms period.

The angle of the servo is controlled mostly by the pulse width (ON time) of the PWM signal. Here,

$$1 \text{ ms ON time} \rightarrow 0^\circ$$

$$1.5 \text{ ms ON time} \rightarrow 90^\circ$$

$$2 \text{ ms ON time} \rightarrow 180^\circ$$

Thus, if both PWM signals have the same ON-time/pulse width, and both operate within the frequency range accepted by the servo, then the two can use different frequencies and still achieve the same approximate servo angle. They may have different duty-cycle percentages due to the different periods they are in use, but servo gets the same position-command pulse width.

4. The L298N loses about 2V internally. If a 12V supply is connected to the driver, roughly what voltage reaches the motor? And if you want a 9V motor to run at close to full speed, what supply voltage should you pick? [Lab 7]

Answer:

The voltage drop inside the L298N is about 2V.

For a 12 V supply:

$$[V_{\text{motor}} = 12\text{V} - 2\text{V} = 10\text{V}]$$

Therefore, motors receive only about 10V.

To have a 9V motor to run at close to full speed, the input voltage should be approximately 2V higher than the motor's rated voltage. And so the voltage should be ,
[$9 + 2 = 11\text{V}$]